

Практическая работа №3

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке

<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Добавлено объявление новых тем в заголовочном файле

```
namespace QSourceHighlighter {  
  
class QSourceHighlighter : public QSyntaxHighlighter  
{  
public:  
    enum Themes {  
        Monokai = 1,  
        DarkTheme = 2,  
        LightTheme = 3,  
    };  
};
```

2) По аналогии с функцией monokai() добавлены две темы, их вызовы добавлены в функцию theme()

```
static QHash<QSourceHighlighter::Token, QTextCharFormat> dark()  
{  
    QHash<QSourceHighlighter::Token, QTextCharFormat> _formats = formats();  
  
    _formats[QSourceHighlighter::Token::CodeBlock].setForeground(QColor(135, 133, 115));  
    _formats[QSourceHighlighter::Token::CodeBlock].setBackground(QColor(34, 34, 27));  
    _formats[QSourceHighlighter::Token::CodeKeyWord].setForeground(QColor(95, 145, 130));  
    _formats[QSourceHighlighter::Token::CodeString].setForeground(QColor(125, 151, 38));  
    _formats[QSourceHighlighter::Token::CodeComment].setForeground(QColor(95, 94, 78));  
    _formats[QSourceHighlighter::Token::CodeType].setForeground(QColor(186, 98, 54));  
    _formats[QSourceHighlighter::Token::CodeOther].setForeground(QColor(157, 108, 124));  
    _formats[QSourceHighlighter::Token::CodeNumLiteral].setForeground(QColor(174, 115, 19));  
    _formats[QSourceHighlighter::Token::CodeBuiltIn].setForeground(QColor(165, 152, 13));  
  
    return _formats;  
}  
  
static QHash<QSourceHighlighter::Token, QTextCharFormat> light()  
{  
    QHash<QSourceHighlighter::Token, QTextCharFormat> _formats = formats();  
  
    _formats[QSourceHighlighter::Token::CodeBlock].setForeground(QColor(34, 34, 27));  
    _formats[QSourceHighlighter::Token::CodeBlock].setBackground(QColor(255, 255, 255));  
  
    _formats[QSourceHighlighter::Token::CodeKeyWord].setForeground(QColor(95, 145, 130));  
    _formats[QSourceHighlighter::Token::CodeString].setForeground(QColor(125, 151, 38));  
    _formats[QSourceHighlighter::Token::CodeComment].setForeground(QColor(95, 94, 78));  
    _formats[QSourceHighlighter::Token::CodeType].setForeground(QColor(186, 98, 54));  
    _formats[QSourceHighlighter::Token::CodeOther].setForeground(QColor(157, 108, 124));  
    _formats[QSourceHighlighter::Token::CodeNumLiteral].setForeground(QColor(174, 115, 19));  
    _formats[QSourceHighlighter::Token::CodeBuiltIn].setForeground(QColor(165, 152, 13));  
  
    return _formats;  
}
```

```

QHash<QSourceHighlighter::Token, QTextCharFormat>
QSourceHighlighterTheme::theme(QSourceHighlighter::Themes theme) {
    switch (theme) {
    case QSourceHighlighter::Themes::Monokai:
        return monokai();
    case QSourceHighlighter::Themes::DarkTheme:
        return dark();
    case QSourceHighlighter::Themes::LightTheme:
        return light();
    default:
        return def();
    }
}

```

и в интерфейс программы

```

void MainWindow::initThemesComboBox()
{
    ui->themeComboBox->addItem("Default", 0);
    ui->themeComboBox->addItem("Monokai", QSourceHighlighter::Themes::Monokai);
    ui->themeComboBox->addItem("Dark Theme", QSourceHighlighter::Themes::DarkTheme);
    ui->themeComboBox->addItem("Light Theme", QSourceHighlighter::Themes::LightTheme);

    ui->themeComboBox->setCurrentIndex(0);
}

```

3)Добавлена функция смены фона при изменении темы

```

478 void MainWindow::themeChanged(int) {
479     QSourceHighlighter::Themes theme = (QSourceHighlighter::Themes)ui->themeComboBox->currentData().toInt();
480     highlighter->setTheme(theme);
481     applyEditorBackground(theme);
482 }
483
484 void MainWindow::applyEditorBackground(QSourceHighlighter::Themes theme) {
485     auto themeFormats = QSourceHighlighterTheme::theme(theme);
486     QTextCharFormat blockFormat = themeFormats.value(QSourceHighlighter::CodeBlock);
487     QColor bgColor = blockFormat.background().color();
488     // if(!bgColor.isValid()) {
489     //     bgColor = "ffffff";
490     // }
491     QString styleSheet = QString("QPlainTextEdit {background-color: %1; }").arg(bgColor.name());
492     ui->plainTextEdit->setStyleSheet(styleSheet);
493     ui->statusbar->showMessage(styleSheet);
494 }
495
496

```

4) при сохранении файла в JSON формате сохраняется тема, при открытии тема подгружается

```

        return;
    }
    QFile file(fileName);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи", 3000);
        return;
    }
    QJsonObject root;
    root["language"] = ui->langComboBox->currentText();
    root["theme"] = ui->themeComboBox->currentData().toInt();
    root["text"] = ui->plainTextEdit->toPlainText();

    QJsonDocument doc(root);
    file.write(doc.toJson());

    file.close();

    isUnnamed = false;
    currentFile = fileName;
    ui->statusbar->showMessage("Файл " + fileName + " сохранен", 3000);
}

QJsonParseError error;
QJsonDocument doc = QJsonDocument::fromJson(file.readAll(), &error);
file.close();
if (error.error != QJsonParseError::NoError) {
    ui->statusbar->showMessage("Ошибка парсинга.", 3000);
    return;
}

QJsonObject obj = doc.object();
QString lang = obj["language"].toString();
int theme = obj["theme"].toInt();
QString text = obj["text"].toString();

ui->plainTextEdit->setPlainText(text);

if (!lang.isEmpty()) {
    ui->langComboBox->setCurrentText(lang);
}

```

Ответы на контрольные вопросы

Как передается значение из QComboBox в слот выбора темы?

Через сигнал currentIndexChanged, который подключается к слоту. В слоте по индексу или строке применяется нужная тема.

Какую роль играет QHash<QSourceHighlighter::Token, QTextCharFormat> в реализации тем оформления?

Хранит соответствие между типом токена (ключевое слово, строка и т.д.) и его визуальным стилем. Позволяет быстро применять форматирование при подсветке и переключать темы целиком.

Какие элементы кода окрашиваются в функции описания темы оформления?

Перечислите все токены и их назначение.

CodeBlock — тело/блок кода (цвет текста и фона)

CodeKeyword — ключевые слова (if, while, return и т.д.)

CodeString — строковые литералы

CodeComment — комментарии

CodeType — типы данных (int, char, void и т.д.)

CodeOther — прочие элементы (операторы, символы)

CodeNumLiteral — числовые литералы (константы, числа)

CodeBuiltIn — встроенные функции/объекты языка

Какой класс в Qt используется для задания форматирования текста, и как он применяется при реализации подсветки синтаксиса через QSourceHighlighter?

Класс QTextCharFormat. Он задаёт цвет, шрифт, стиль. В QSyntaxHighlighter создаётся нужный формат, затем setFormat(позиция, длина, формат) применяет его к фрагменту текста.

Как сохранить текущую тему оформления в JSON-файле вместе с текстом и языком программирования?

В JSON добавляется поле "theme", где указывается текущая тема. При загрузке это поле парсится, применяется тема и вызывается rehighlight().